src/llm.py

The Claude boundary — one wrapper, with an on-disk response cache

Every call to the Anthropic API goes through this module. It caches each response by a hash of its inputs (so notebooks and tests reproduce numbers without a key) and degrades gracefully when no key is present, raising a clear, catchable error instead of a confusing SDK crash.

Walkthrough of all 160 lines, in order. Left numbers are the real line numbers in the file.

```
 1  """Claude wrapper with an on-disk response cache — the project's LLM boundary.
 2
 3  Everything that calls the Anthropic API goes through here. Two jobs:
 4
 5  1. **Cache every response by a hash of its inputs.** The cache is committed to
 6     `outputs/results/llm_cache.json`, so the notebook, the tests, and
 7     `test_pipeline_integrity.py` reproduce every published number *without an API
 8     key* — exactly the "reference-only, load the saved result" convention the
 9     portfolio's Kaggle-GPU notebooks use. Re-running the pipeline also never pays
10     for an identical call twice.
11
12  2. **Degrade gracefully with no key.** If `ANTHROPIC_API_KEY` is unset and a call
13     isn't already cached, we raise a clear, catchable error rather than a confusing
14     SDK exception — the caller can then decide to skip that step.
15
16  The model id, token cap, and cache path are all named constants in config.py.
17  Uses the official `anthropic` SDK (not raw HTTP) with adaptive thinking, per the
18  current Claude API guidance.
19  """
```

Lines 1 to 19 are the module docstring. They state the two jobs: cache every response by an input hash (committed so results reproduce without a key, and never paying twice for an identical call), and degrade gracefully with no key by raising a clear error. They also note it uses the official anthropic SDK with adaptive thinking.

```
20
21  from __future__ import annotations
22
23  import functools
24  import hashlib
25  import json
26  from typing import Any
27
28  from . import config
```

Lines 20 to 28 are imports: postponed annotations (line 21), functools for the caching decorators (line 23), hashlib for the input hash (line 24), json for cache serialization (line 25), Any for type hints (line 26), and the project config (line 28).

```
29
30
31  class LLMKeyMissing(RuntimeError):
32      """Raised when a live call is needed but no API key is available.
33
34      Distinct exception type so callers (the eval harness, ask.py) can catch it
35      and fall back to "reproduce from cache only" instead of crashing.
36      """
37
```

Lines 29 to 37 define LLMKeyMissing, a custom exception subclassing RuntimeError. The docstring explains the point of a distinct type: callers such as the eval harness and ask.py can catch exactly this and fall back to reproduce-from-cache instead of crashing.

```
38
39  # ---------------------------------------------------------------------- #
40  # Cache
41  # ---------------------------------------------------------------------- #
42  @functools.lru_cache(maxsize=1)
43  def _load_cache() -> dict[str, Any]:
44      """Load the on-disk response cache (once per process)."""
45      if config.LLM_CACHE_FILE.exists():
46          with open(config.LLM_CACHE_FILE, encoding="utf-8") as f:
47              return json.load(f)
48      return {}
```

Lines 38 to 48 define _load_cache. The @functools.lru_cache(maxsize=1) decorator (line 42) means the cache file is read from disk only once per process. Lines 45 to 47 open and JSON-load the cache file if it exists; line 48 returns an empty dict otherwise, so a first run with no cache still works.

```
49
50
51  def _save_cache(cache: dict[str, Any]) -> None:
52      """Persist the response cache."""
53      config.ensure_dirs()
54      with open(config.LLM_CACHE_FILE, "w", encoding="utf-8") as f:
55          json.dump(cache, f, indent=2, ensure_ascii=False)
56
57
```

Lines 49 to 57 define _save_cache, which persists the cache dict back to disk. Line 53 ensures the output directory exists; lines 54 to 55 write the cache as indented JSON with ensure_ascii=False so non-ASCII text is stored readably.

```
58  def _cache_key(payload: dict[str, Any]) -> str:
59      """Deterministic hash of the request payload.
60
61      Sorting keys makes the hash independent of dict ordering, so the same logical
62      request always maps to the same cache entry.
63      """
64      blob = json.dumps(payload, sort_keys=True, ensure_ascii=False)
65      return hashlib.sha256(blob.encode("utf-8")).hexdigest()
```

Lines 58 to 65 define _cache_key, the deterministic hash of a request. Line 64 dumps the payload to JSON with sort_keys=True so dictionary ordering never affects the hash; line 65 returns the SHA-256 hex

digest of those bytes. Identical logical requests therefore map to the same cache entry.

```
66
67
68  # ---------------------------------------------------------------------------- #
69  # Client
70  # ---------------------------------------------------------------------------- #
71  @functools.lru_cache(maxsize=1)
72  def _get_client():
73      """Construct the Anthropic client (lazy — only when a live call happens)."""
74      import anthropic
75
76      return anthropic.Anthropic()
```

Lines 66 to 76 define _get_client, which constructs the Anthropic SDK client. It is also lru_cache'd (line 71) so the client is built once, and the import of anthropic is inside the function (line 74) so it only happens when a live call is actually needed — keeping key-free runs import-light.

```
77
78
79  def _blocks_to_dicts(content) -> list[dict[str, Any]]:
80      """Serialize Claude response content blocks to plain dicts for caching."""
81      out: list[dict[str, Any]] = []
82      for block in content:
83          if block.type == "text":
84              out.append({"type": "text", "text": block.text})
85          elif block.type == "tool_use":
86              out.append(
87                  {"type": "tool_use", "id": block.id, "name": block.name, "input": block.input}
88              )
89          # thinking blocks carry no content under the default display; skip them.
90      return out
```

Lines 77 to 90 define _blocks_to_dicts, which converts Claude's response content blocks into plain dictionaries that survive JSON caching. It loops over the blocks (line 82); text blocks become a type/text dict (lines 83 to 84); tool_use blocks keep their id, name, and input (lines 85 to 88); thinking blocks carry no displayable content and are skipped (line 89). Line 90 returns the list.

```
 91
 92
 93  def call(
 94      messages: list[dict[str, Any]],
 95      system: str | None = None,
 96      tools: list[dict[str, Any]] | None = None,
 97      model: str = config.GEN_MODEL,
 98      max_tokens: int = config.MAX_ANSWER_TOKENS,
 99      use_cache: bool = True,
100  ) -> dict[str, Any]:
101      """Call Claude once and return a plain-dict response.
102
103      The return shape is `{"stop_reason": str, "content": [blocks...]}` where each
104      block is a serialized dict — everything the agent loop and the answer parser
105      need, and everything that round-trips cleanly through the JSON cache.
106
107      On a cache hit we never touch the network (so no key is needed). On a miss
108      with no key we raise `LLMKeyMissing` so the caller can fall back.
109      """
110      payload = {
111          "model": model,
112          "max_tokens": max_tokens,
113          "system": system,
114          "messages": messages,
115          "tools": tools,
116      }
```

Lines 91 to 116 begin call(), the one function everything uses. Its parameters (lines 93 to 100) are the messages, an optional system prompt, optional tools, and the model, max_tokens, and use_cache flag defaulting from config. The docstring (lines 101 to 109) documents the return shape (a stop_reason plus serialized content blocks) and the cache-hit and key-missing behaviours. Lines 110 to 116 assemble the payload dict (model, tokens, system, messages, tools) and — with line 117 — hash it into a cache key.

```
117      key = _cache_key(payload)
118
119      if use_cache:
120          cache = _load_cache()
121          if key in cache:
122              return cache[key]
123
```

Lines 117 to 123 handle the cache lookup. Line 117 computes the key. When caching is on (line 119) it loads the cache (line 120) and, on a hit (line 121), returns the stored response immediately (line 122) — no network, no key required.

```
124      if not config.api_key_available():
125          raise LLMKeyMissing(
126              "No ANTHROPIC_API_KEY set and this request is not cached. "
127              "Set the key to generate it, or run only the cached/local steps."
128          )
```

Lines 124 to 128 are the graceful-degradation path: if no API key is available (line 124), and we reached here, the request was not cached, so it raises LLMKeyMissing with a message telling the user to set the key or run only cached/local steps.

```
129
130     client = _get_client()
131     request: dict[str, Any] = {
132         "model": model,
133         "max_tokens": max_tokens,
134         "messages": messages,
135         # Adaptive thinking: Claude decides how much to reason per request.
136         "thinking": {"type": "adaptive"},
137     }
138     if system is not None:
139         request["system"] = system
140     if tools is not None:
141         request["tools"] = tools
```

Lines 129 to 141 build the live request. Line 130 gets the cached client. Lines 131 to 137 assemble the request with model, tokens, messages, and adaptive thinking (letting Claude decide how much to reason). Lines 138 to 141 add the system prompt and tools only if they were provided, keeping the request minimal otherwise.

```
142
143     response = client.messages.create(**request)
144     result = {
145         "stop_reason": response.stop_reason,
146         "content": _blocks_to_dicts(response.content),
147     }
148
```

Lines 142 to 148 execute the call. Line 143 sends the request via client.messages.create. Lines 144 to 147 build the plain-dict result: the response's stop_reason and its content run through _blocks_to_dicts so it is JSON-serializable.

```
149     if use_cache:
150         cache = _load_cache()
151         cache[key] = result
152         _save_cache(cache)
153         _load_cache.cache_clear()  # so the next read sees the write
154
155     return result
```

Lines 149 to 155 write the result to the cache when caching is on: reload the cache (line 150), store the result under the key (line 151), save it (line 152), and clear the lru_cache (line 153) so the next _load_cache sees the new entry. Line 155 returns the result.

```
156
157
158 def first_text(result: dict[str, Any]) -> str:
159     """Extract the concatenated text of a response (ignoring tool_use blocks)."""
160     return "".join(b["text"] for b in result["content"] if b["type"] == "text").strip()
```

Lines 156 to 160 define first_text, a small helper that concatenates the text of every text block in a response (ignoring tool_use blocks) and strips whitespace — the convenient way for rag.py and agent.py to get the plain answer string.